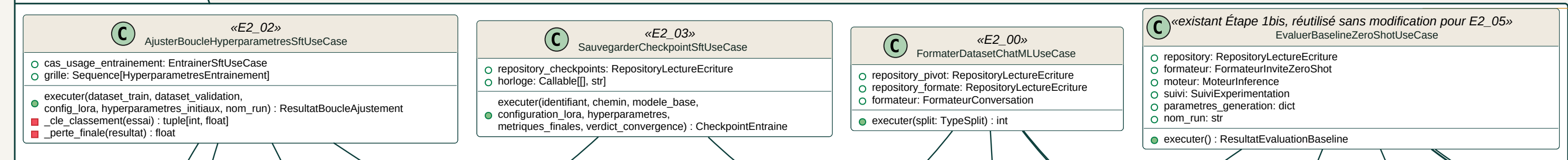
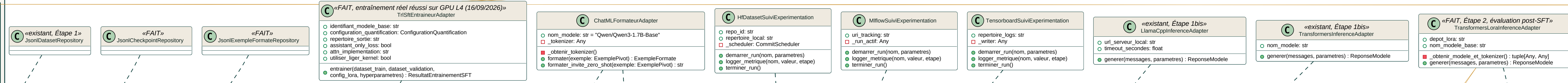


application.use_cases



infrastructure.adapters



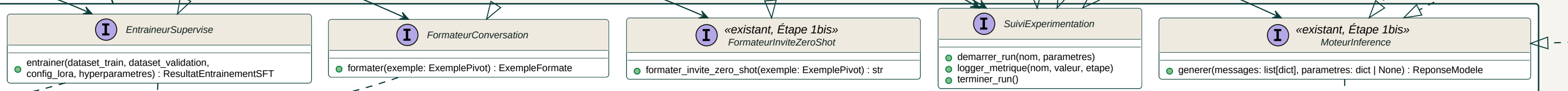
Aucun nouveau cas d'usage écrit pour l'évaluation post-SFT (E2_05) : ce cas d'usage de l'Étape 1bis est agnostique au modèle injecté (seulement un MoteurInference), donc réutilisé tel quel une 3e fois (après CPU/GGUF puis GPU/bf16 zero-shot), seul l'adaptateur (TransformersLoraInferenceAdapter) change. Cf. README §2.4.

Écrit et d'abord vérifié SANS GPU, puis RÉELLEMENT exécuté sur un job HF Jobs GPU L4 facturé le 16/09/2026 : ~20 min, 342 pas, verdict SAINE, poids publiés avec succès sur mombasstic/cha-triage-sft-lora (cf. AGENTS.md, README §2.3). Deux trouvailles réelles sur ce run : entraînement.type_perte=chunked_nll est garanti d'échouer (tri plafonné à 0.24.0 par la dépendance unsloth non câblée), corrigé à nll ; et suivi.backend devait valoir hf_dataset pour que -suivi-hf-repo ne soit pas ignoré en silence (la 1re courbe a été perdue, puis reconstruite à partir du log brut du job par monitoring/reconstruire_courbe_sft_depuis_log.py). Charge le modèle quantifié et construit LoraConfig dans son constructeur, jamais exposé au-dessus de la frontière domaine/application.

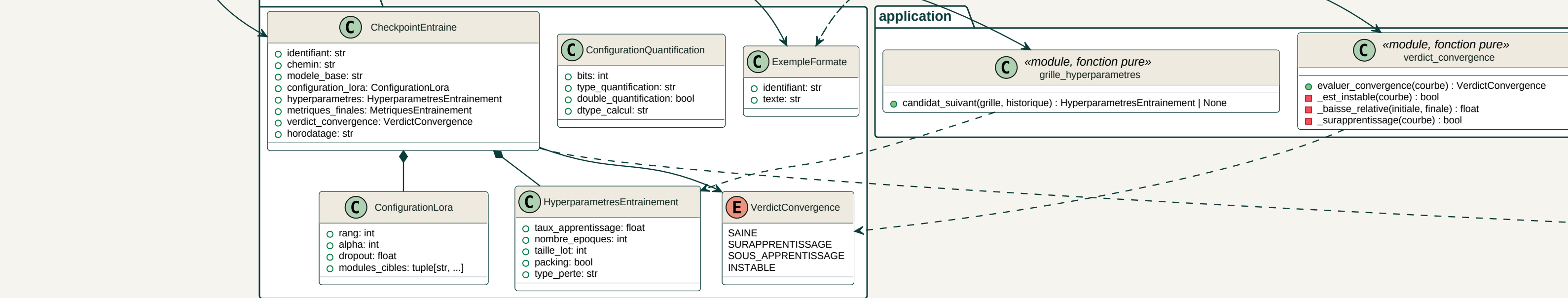
Copie quasi conforme de TransformersInferenceAdapter (charge le modèle de base bf16 PUIS peft.PeftModel.from_pretrained(depot_lora) par-dessus) ; réutilise directement sa fonction _parametres_generation_transformers (jamais dupliquée). Échoue explicitement (RuntimeError) sans GPU CUDA, jamais de repli silencieux vers le CPU. Câblage vérifié sans GPU réel (17/09/2026, cf. README §2.4) ; les vrais chiffres post-SFT restent à produire sur un job HF Jobs GPU réel.

Réutilisé pour 3 types d'entités au total : ExemplePivot (Étape 1), ExempleFormate et CheckpointEntraîne (Étape 2) ; aucune méthode ajoutée au port existant. Chaque entité a son propre adaptateur JSONL dédié (jamais une généralisation de JsonDatasetRepository).

domain.ports



domain.model



application

